Cómo se unificó V3Integracion1.0

Arquitectura, adaptación de interfaz, conservación de algoritmos y evidencia de validación.

Este informe explica la integración realizada sobre una copia de PrototipoV3. Describe qué componentes se reutilizaron, cómo se conectaron, qué archivos cambiaron y qué capacidades siguen pendientes.

Componente	Resultado de esta entrega
Cotidiano	Motor YAMNet + clasificador propio integrado; cuatro contextos en la interfaz.
Música	Biblioteca AudioToHaptics incorporada y conectada a su API por archivo. Demucs sigue ausente.
Háptica	Se reutiliza BLE para los patrones cotidianos existentes. No se añadió firmware musical multicanal.
Verificación	43 pruebas superadas y 6 omitidas por recursos externos ausentes.

Criterio de trabajo: unificar lo entregado sin completar los modelos, sin entrenar nuevas clases y sin modificar los algoritmos de clasificación o síntesis. Las dos excepciones de código se documentan en la sección de cambios: cierre de recursos y disponibilidad de datos de una prueba.

Proyecto: V3/V3Integracion1.0
Versión de la app: 3.0-integracion1.0
Documento basado en código, capturas e informes locales de esta entrega.

INFORME TÉCNICO / 01

Guía de lectura y alcance

Página	Contenido
3	Arquitectura y estructura del proyecto
4	Compilación, dependencias y recursos
5	Funcionamiento del motor cotidiano
6	Contextos y capacidades reales
7	Sesiones, micrófono y ciclo de vida
8	Alertas, Bluetooth y firmware
9	Funcionamiento de la biblioteca musical
10	Conexión musical con la aplicación
11	Interfaz cotidiana: capturas y decisiones
12	Interfaz de Inicio y Música
13	Cambios directos en las fuentes de los equipos
14	Pruebas y auditoría
15	Cómo compilar, utilizar y continuar la entrega
16	Mapa de archivos y límites finales

Qué significa aquí «integrar»

Se creó una aplicación Android que contiene ambas bibliotecas y utiliza sus API reales. La interfaz presenta sus estados y sus capacidades, en lugar de anunciar funciones que los archivos recibidos no pueden ejecutar. Cotidiano dispone de sus modelos; Música está integrada a nivel de código, compilación e interfaz, pero su análisis completo sigue bloqueado por la ausencia de Demucs.

Las carpetas fuente Audio y Cotidiano permanecieron sin editar durante esta integración. PrototipoV3 también se conservó; las adaptaciones de esta etapa están dentro de V3Integracion1.0.

Las referencias del informe son rutas relativas a la raíz de V3Integracion1.0, salvo cuando se indica expresamente un origen externo. Las pruebas del emulador no equivalen a una validación del brazalete físico.

INFORME TÉCNICO / 02

Arquitectura unificada

La aplicación se organizó en tres responsabilidades: presentación, coordinación y motores. Compose muestra controles y resultados; IntegracionViewModel administra las sesiones y las salidas; los módulos de los equipos ejecutan el reconocimiento o el procesamiento musical.



Estructura principal

```
V3Integracion1.0/
  app/                      Interfaz y coordinación
  sound-engine/             Motor cotidiano recibido
  audioToHaptics/            Biblioteca musical recibida
  audioToHapticsModels/      Empaquetado opcional de Demucs
  models/                   Ubicación del modelo pendiente
  firmware/                 Firmware conservado de la app
  docs/                     Cambios, pruebas y capturas
  scripts/                  Auditoría e informe PDF
```

Origen cotidiano: Cotidiano/sound-recognition-main/sound-recognition-main/Android/sound-engine.
Origen musical: Audio/Tesis-main/Tesis-main/AudioToHaptics/audioToHaptics y audioToHapticsModels.

Se eligieron los componentes Android ya existentes. Los scripts Python de entrenamiento, exportación y experimentación no se ejecutan dentro de la app. El motor cotidiano mantiene su propia captura; la biblioteca musical conserva su decodificación por URI. No se forzó a ambos a pasar por el antiguo contrato de bloques mono de V3.

INFORME TÉCNICO / 03

Compilación y recursos

Sección	Adaptación	Motivo
settings.gradle.kts	Incluye sound-engine y audioToHaptics. El módulo de modelos es condicional.	Un solo proyecto que compila aunque Demucs no se haya entregado.
Gradle raíz	Plugins Android library y serialización Kotlin alineados con la app.	Compatibilidad de las bibliotecas con AGP 9.
app/build.gradle.kts	minSdk 26, Java 17, dependencias a módulos y ONNX sin comprimir.	Cumplir requisitos existentes de la biblioteca musical.
sound-engine	Coroutines 1.9.0 y LiteRT 1.4.2 con la API Interpreter.	Reutilizar el runtime de V3 y evitar duplicados TensorFlow Lite.
audioToHaptics	Retiro del plugin Kotlin Android y kotlinOptions antiguos.	Usar Kotlin incorporado en AGP 9, conservando Java 17.

Resolución del conflicto entre modelos YAMNet

La app anterior y el módulo recibido contenían un archivo llamado yamnet.tflite. La implementación anterior esperaba categorías como salida; el equipo cotidiano utiliza la salida de embeddings de 1024 componentes y un segundo clasificador. No son intercambiables por compartir el nombre.

Se retiró el yamnet.tflite del directorio de assets de la copia de la app. El APK recibe el archivo original de sound-engine, junto con head.tflite, clases.txt y contexts.json. Los archivos de ese módulo no fueron renombrados ni reentrenados.

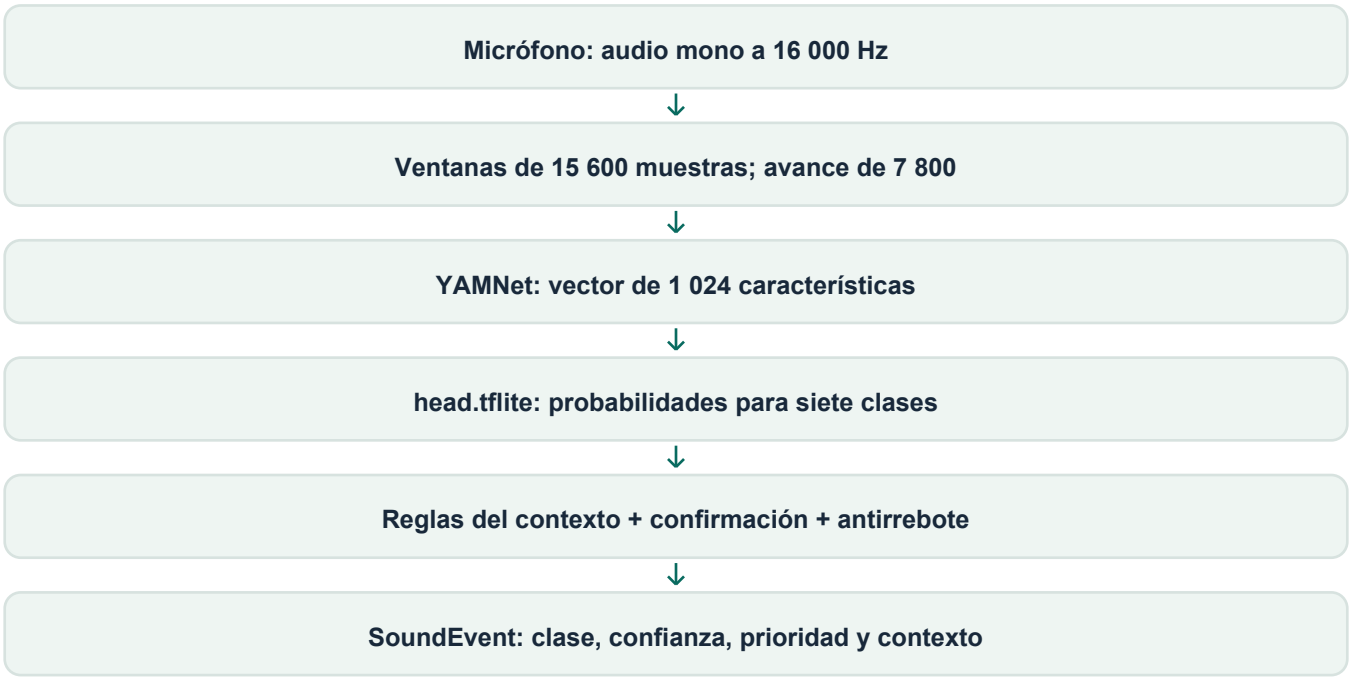
Paquete opcional musical

settings.gradle.kts incluye audioToHapticsModels únicamente si existe models/htdemucs_6s.onnx. Así se conserva la comprobación original de SHA-256 cuando el archivo se entregue, sin bloquear la compilación actual ni simular que el modelo está instalado.

La obtención de dependencias por Gradle durante la compilación es distinta de descargar modelos al usar la app. Esta integración no generó ni descargó Demucs.

INFORME TÉCNICO / 04

Motor cotidiano: funcionamiento



Cada ventana representa aproximadamente 0,975 segundos y avanza 0,4875 segundos. El motor no clasifica solo el nivel de volumen: YAMNet extrae características del sonido y el clasificador entrenado produce las probabilidades de las clases del proyecto.

Reglas conservadas

Una clase debe estar habilitada en el contexto y superar su umbral. El motor comprueba el nivel mínimo configurado y exige que la misma clase aparezca en dos ventanas válidas consecutivas. Limita las alertas repetidas de una misma clase con un antirrebote de tres segundos.

La integración consume los eventos ya confirmados. No vuelve a aplicar el umbral global 0,35 ni el intervalo global del coordinador anterior, porque eso introduciría una segunda política distinta de las reglas del equipo.

Información mostrada

Al llegar un evento, se presenta la clase con un nombre legible, su confianza, prioridad y contexto. También se conserva un historial de hasta 30 detecciones, del que la pantalla muestra las ocho más recientes.

El motor calcula un nivel acústico estimado con un ajuste pendiente de calibración por teléfono. La interfaz informa esa limitación; no lo presenta como una medición fiable de distancia.

INFORME TÉCNICO / 05

Contextos y clases disponibles

CatalogoCotidianoEquipo lee los mismos contexts.json y clases.txt que usa SoundEngine. Una clase configurada solo se muestra como disponible si aparece entre las etiquetas del clasificador. No se añadieron capacidades cambiando textos de la interfaz.

Contexto	Sonido	Prior.	Umbral	Modelo
Hogar	Alarma de humo	3	50 %	Pendiente
Hogar	Llanto de bebé	3	55 %	Disponible
Hogar	Timbre	2	60 %	Pendiente
Hogar	Vidrio roto	3	60 %	Disponible
Hogar	Ladrido	1	75 %	Disponible
Hogar	Nombre del usuario	3	70 %	Pendiente
Urbano	Sirena	3	60 %	Disponible
Urbano	Claxon	3	60 %	Disponible
Urbano	Nombre del usuario	2	75 %	Pendiente
Social	Nombre del usuario	3	70 %	Pendiente
Social	Alarma de humo	3	55 %	Pendiente
Social	Sirena	2	65 %	Disponible
Industrial	Maquinaria	3	60 %	Disponible
Industrial	Alarma de humo	3	50 %	Pendiente
Industrial	Sirena	2	65 %	Disponible
Industrial	Nombre del usuario	2	75 %	Pendiente

Clases entrenadas: claxon, fondo, ladrido, llanto_bebe, maquinaria, sirena y vidrio_roto. Fondo no se presenta como alerta de un contexto. Alarma de humo, timbre y nombre del usuario siguen pendientes de entrenamiento.

El contexto **Social** del modelo recibido permite detectar sirena; no se cambió para convertirlo en el antiguo perfil social del firmware. Son configuraciones distintas y la interfaz las separa.

INFORME TÉCNICO / 06

Sesiones y ciclo de vida

Inicio de la escucha

La pantalla solicita permiso de micrófono. Si se concede mientras Cotidiano está visible y la aplicación está en un estado activo, el coordinador puede iniciar una sesión. Se evita iniciar dos sesiones a la vez o comenzar la escucha mientras se realiza un análisis musical.

Dentro de un Mutex se crea el motor del equipo en un hilo de trabajo, se selecciona el contexto y se registran los colectores de eventos y log antes de llamar a start. Esto importa porque el flujo de eventos no conserva un historial para suscriptores tardíos.

Cambio de contexto

Elegir Hogar, Urbano, Social o Industrial detiene la sesión actual, invalida eventos anteriores y actualiza los casos visibles. El usuario pulsa de nuevo Iniciar escucha. Esta decisión evita modificar el mapa de reglas del motor mientras una inferencia lo está leyendo en otro hilo.

Cancelación y liberación

La sesión se mantiene suspendida hasta cancelarse. Al salir de Cotidiano o detener la escucha, la tarea se cancela y el bloque finally espera releaseAndJoin en un contexto NonCancellable. El Mutex impide que la siguiente sesión abra otro micrófono antes de que termine esa limpieza.

Mecanismo	Función
exclusividadEscucha	Serializa las sesiones y la liberación de recursos.
versionEscucha	Descarta resultados de una sesión que ya se detuvo.
CoroutineStart.UNDISPATCHED	Registra los colectores antes de arrancar la captura.
NonCancellable + Dispatchers.Default	Completa el cierre nativo aun si la pantalla canceló la sesión.

El micrófono utilizado es el de SoundEngine. La ruta activa integrada no abre simultáneamente el AudioRecord del adaptador antiguo EntradaAudioAndroid. Al detener la pantalla también se detienen las señales y las vistas previas administradas por la aplicación.

INFORME TÉCNICO / 07

Alertas y conexión háptica

Se conservó ConexionEntornoBle y los comandos de los firmware incluidos. El coordinador traduce únicamente las clases que ya tienen una correspondencia existente y verifica el perfil de firmware elegido por el usuario antes de transmitir.

Clase detectada	Comando	Perfil admitido	Si no se puede enviar
ladrido	DOG	Social	Patrón existente en el teléfono.
llanto_bebe	BABY	Social	Patrón existente en el teléfono.
sirena	SIREN	Alerta bicicleta	Patrón existente en el teléfono.
claxon	HORN	Alerta bicicleta	Patrón existente en el teléfono.
maquinaria	Sin comando	No definido	Aviso visual, sin patrón inventado.
vidrio_roto	Sin comando	No definido	Aviso visual, sin patrón inventado.

Contexto acústico y firmware son independientes

El contexto decide qué sonidos intenta reconocer el modelo y con qué reglas. El perfil de firmware indica qué órdenes entiende el ESP32. Por ejemplo, Hogar puede detectar un ladrido, pero enviar DOG solo tiene sentido si está seleccionado el firmware Social.

Indicadores de Inicio

Bluetooth muestra si el adaptador del teléfono está activado, desactivado, no disponible o pendiente de permiso. Háptica refleja el flujo de conexión del coordinador: desconectada, conectando o conectada. El estado positivo requiere descubrir el servicio y la característica de escritura esperados.

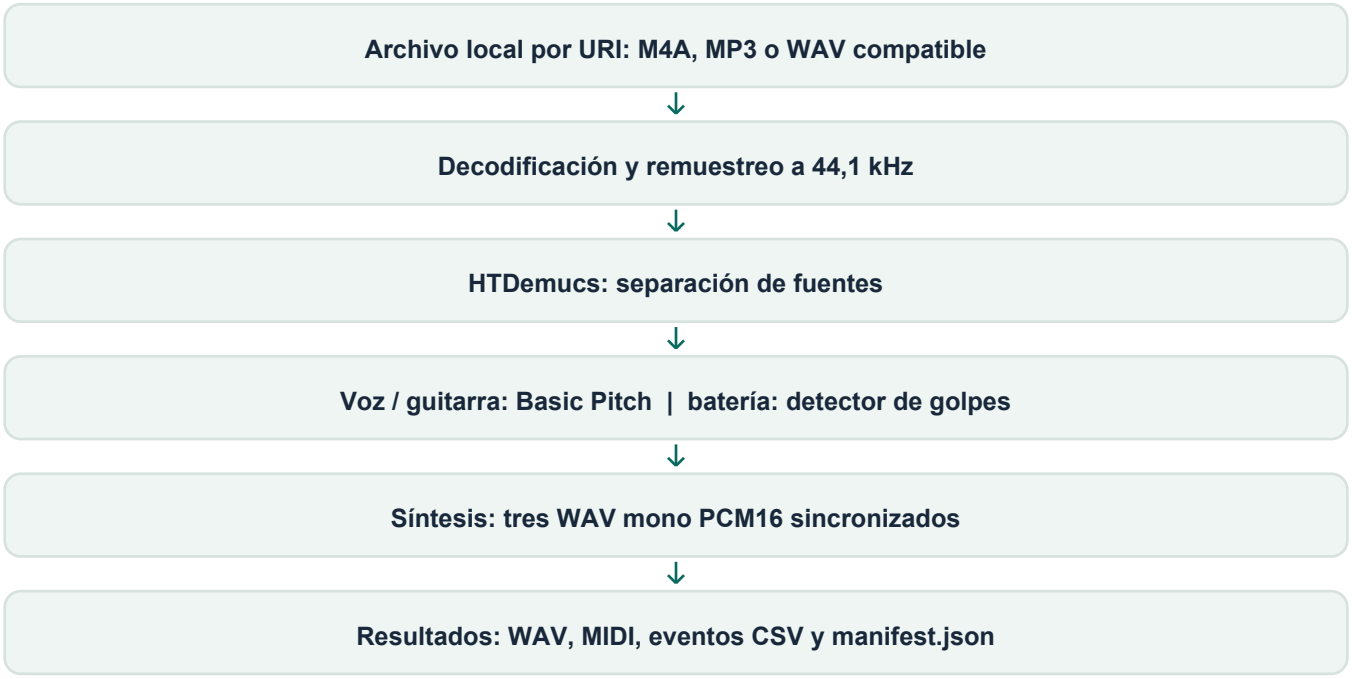
Los firmware comparten nombre y UUID. El descubrimiento GATT no identifica automáticamente cuál está instalado ni garantiza que un actuador haya vibrado. La confirmación de escritura tampoco mide una ejecución física. No se ensayó el brazalete en esta entrega.

Servicio: 12345678-1234-1234-1234-1234567890ab
Comandos: abcdefab-1234-5678-1234-abcdefabcdef
Nombre anunciado: VIBRAFIL_ESP32

INFORME TÉCNICO / 08

Biblioteca musical recibida

Se reutilizó la implementación Android de AudioToHaptics, con su decodificación, ONNX Runtime y procesamiento DSP. Su código de producción permanece idéntico al recibido. El siguiente flujo describe lo que ejecuta la biblioteca cuando están presentes todos los recursos; no acredita que se haya ejecutado aquí el análisis completo.



Qué representan los canales

Voz, guitarra y percusión se convierten en señales con una portadora fija por actuador. Se representan la duración y el ritmo de los eventos y, según configuración, su intensidad. La altura de una nota no se convierte en una frecuencia de vibración distinta para cada nota.

Recursos presentes y ausentes

Basic Pitch está incluido como basic_pitch_nmp.onnx. Falta htdemucs_6s.onnx, necesario para separar los instrumentos. modelStatus permite mostrar ese estado antes de ofrecer un análisis que no puede completarse.

La integración contiene la biblioteca y su ruta de ejecución, pero **no generó, descargó ni sustituyó Demucs**. La ausencia del modelo se conserva como pendiente explícito.

INFORME TÉCNICO / 09

Conexión musical con la app

Por qué se usa la API por URI

La interfaz anterior AlgoritmoMusica.procesar recibía fragmentos de audio mono. La biblioteca recibida procesa una canción completa mediante AudioToHaptics.process(uri), administra su decodificación y separa fuentes. Se conectó esa API al selector de archivos existente, en lugar de modificar la biblioteca para adaptarla al contrato anterior.

Condiciones del botón Analizar música

Se exige que el reproductor tenga un archivo listo, que ambos modelos estén incluidos, que el usuario haya activado la configuración de prueba y que no haya otro análisis en curso. En esta entrega el modelo faltante mantiene el botón deshabilitado y se muestra la explicación.

Configuración, progreso y errores

Los parámetros oficiales de actuadores siguen pendientes. El usuario puede seleccionar explícitamente HapticsConfig.testValues: 150/200/250 Hz y 44,1 kHz. Son los valores de laboratorio originales, sin confirmación para el hardware. También puede pedir que se guarden las fuentes separadas.

El coordinador ejecuta una tarea cancelable, evita dos análisis simultáneos y muestra etapas en español y porcentaje. Controla errores de configuración, modelo ausente, formato, procesamiento y memoria. Un contador de versión evita que un análisis cancelado actualice una selección posterior.

Resultados y reproducción

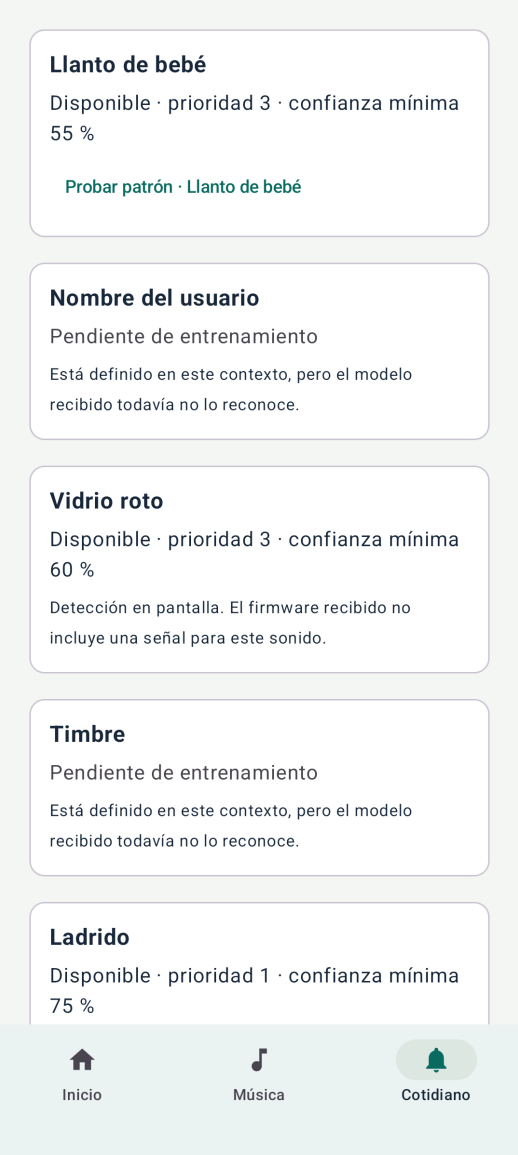
La pantalla contempla los tres ActuatorOutput: cuenta de eventos, portadora, WAV, MIDI cuando corresponda y CSV. También muestra advertencias y la carpeta de resultados. La vista previa escucha los archivos mediante MediaPlayer, como la demo original. Al empezar una vista previa se detiene la escucha ambiental.

> No se conectaron los tres canales al secuenciador de notas antiguo: ese secuenciador descarta eventos simultáneos y no representa canales independientes. Tampoco se inventó una transmisión de WAV por el protocolo BLE actual.

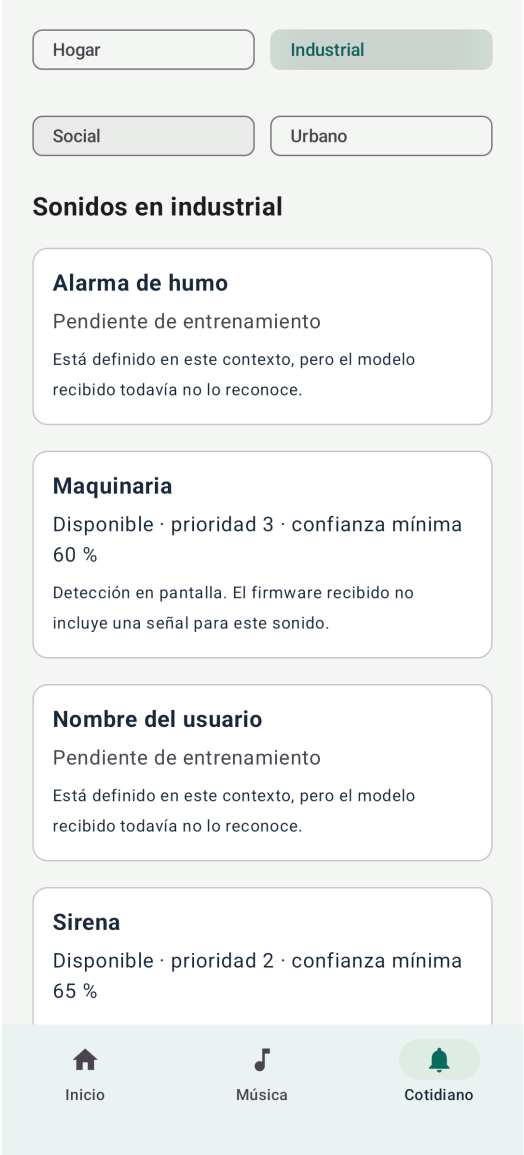
INFORME TÉCNICO / 10

Interfaz cotidiana: evidencia

Las capturas se obtuvieron durante las pruebas instrumentadas. Las tarjetas proceden del catálogo del algoritmo: disponibilidad, prioridad y umbral. Las clases sin patrón de firmware tienen una explicación visual; las pendientes no se anuncian como reconocibles.



Hogar. Llanto de bebé y vidrio roto disponibles; nombre del usuario y timbre pendientes. El botón de prueba representa un patrón manual, no una detección.



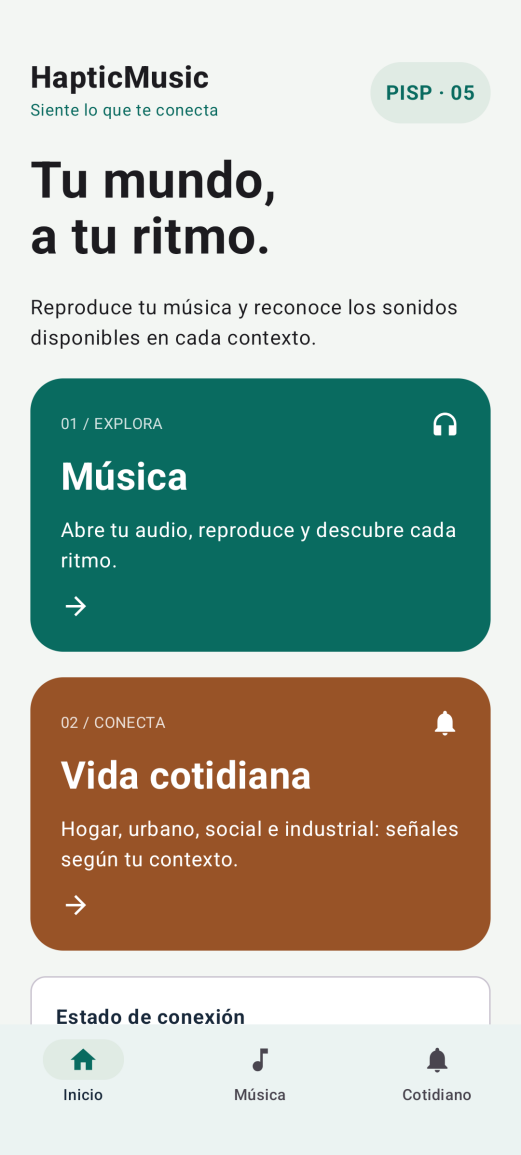
Industrial. La lista cambia con el contexto. Maquinaria se presenta como detección visual porque el firmware recibido no tiene un comando para ella.

La sección del dispositivo permite elegir el firmware de salida independientemente del contexto. La intensidad del teléfono ajusta los patrones locales; la intensidad del brazalete depende de su firmware.

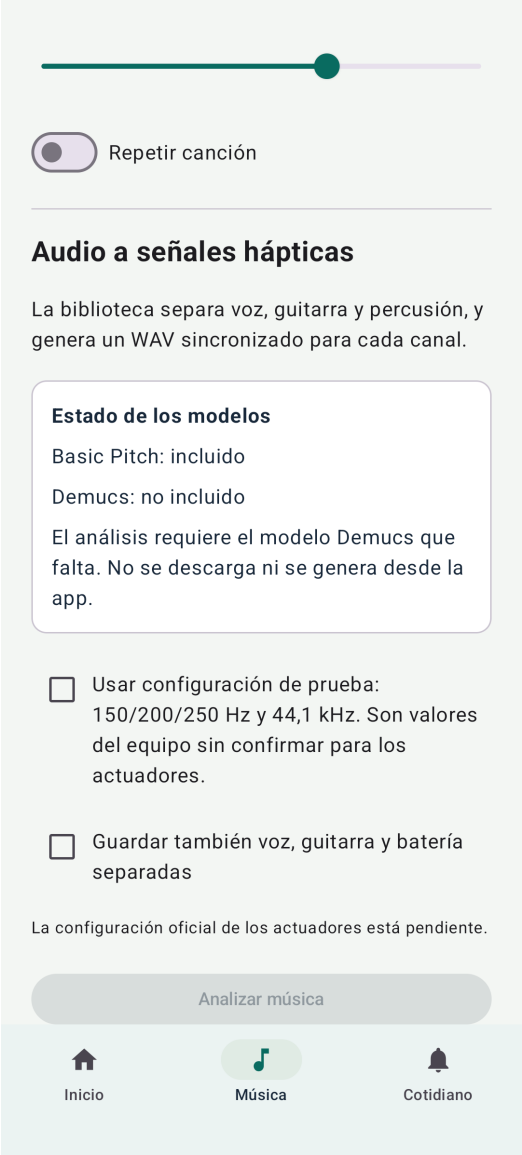
INFORME TÉCNICO / 11

Inicio y Música: evidencia

La navegación conserva Inicio, Música y Cotidiano. No reaparece el juego musical eliminado en PrototipoV3. La pantalla de Música distingue reproducción normal, presencia de modelos y configuración para analizar.



Inicio. Acceso a las experiencias y al estado de conexión Bluetooth/háptica dentro del panel desplazable.



Música. Basic Pitch incluido, Demucs ausente y configuración de prueba identificada. El análisis está deshabilitado por sus requisitos pendientes.

La captura musical corresponde a la app compilada sin Demucs. No se utilizaron imágenes de resultados ficticios para aparentar que ya se habían separado instrumentos.

INFORME TÉCNICO / 12

Cambios directos en los equipos

1. SoundEngine.kt: releaseAndJoin

Ubicación: sound-engine/src/main/java/com/indoamerica/haptica/soundengine/SoundEngine.kt, método añadido junto a stop y release.

```
suspend fun releaseAndJoin() {
    val actual = job
    actual?.cancelAndJoin()
    job = null
    scope.cancel()
    yamnet.close()
    head.close()
}
```

Motivo: release cancelaba la tarea y cerraba los intérpretes inmediatamente. Una lectura o inferencia podía seguir terminando. El nuevo método espera el final de la tarea antes de liberar los modelos y permite serializar el cambio de sesión. La app no llama a stop antes de este método, para conservar la referencia que debe esperar.

Alcance: gestión de recursos. No modifica inferencia, ventanas, pesos, clases, umbrales, prioridades ni antirrebote. Los métodos originales siguen disponibles.

2. ModelIntegrationTest.kt: datos de referencia

Ubicación: audioToHaptics/src/test/kotlin/com/tesis/audiotohaptics/ModelIntegrationTest.kt, inicio de basicPitchKotlinMatchesNumpyReferenceAndPrototype.

Se añadió una comprobación Assume.assumeTrue de que exista al menos una pareja de stem WAV y referencia CSV. Sin esos archivos, la prueba original saltaba sus entradas, producía un informe vacío y terminaba fallando. Ahora se registra como omitida por datos ausentes.

Cuando las referencias existen se conservan las comparaciones numéricas originales. No cambió el modelo Basic Pitch ni su código de producción. Los otros dos cambios en los módulos son ajustes de sus scripts Gradle, explicados en la página 4.

Las diferencias exactas, archivo por archivo, están en docs/auditoria/. El registro ampliado está en docs/CAMBIOS_INTEGRACION.md.

INFORME TÉCNICO / 13

Pruebas y conservación de fuentes

Grupo	Superadas	Omitidas	Fallos
Aplicación / JVM	11	0	0
Biblioteca musical / JVM	25	6	0
Emulador Android 17 / API 37	7	0	0
Total	43	6	0

Qué se comprobó en el emulador

Se verificó la coincidencia del catálogo con los assets; una inferencia real YAMNet con salida de 1024 componentes; la salida de siete probabilidades finitas de head.tflite; y dos sesiones consecutivas de micrófono con cierre seguro. También se comprobó la navegación entre contextos y el aviso del modelo musical ausente.

La inferencia utiliza silencio para comprobar carga, dimensiones y ejecución. No es una medición de exactitud acústica. Las pruebas musicales disponibles validan configuración, DSP, síntesis y errores con recursos locales o sintéticos.

Por qué se omitieron seis pruebas

Faltan recursos externos para la referencia numpy del remuestreador, la comparación Basic Pitch, la comparación Demucs, la percusión de una batería real y los dos pipelines de canciones completas. No se contaron como pruebas superadas.

Auditoría reproducible

Comparación de módulos	Resultado
Archivos comparados	47
Archivos idénticos por bytes	43
Archivos modificados	4: dos Gradle, un método de cierre y una condición de prueba
Otros cambios en producción	Ninguno detectado por el script

scripts/verificar_fuentes.py calcula SHA-256 contra las carpetas originales y genera JSON y diffs. Todas las fuentes de producción de audioToHaptics y los assets de los motores permanecen idénticos. Esta auditoría de 47 archivos corresponde a los módulos copiados, no a todos los archivos de la aplicación.

INFORME TÉCNICO / 14

Uso y reproducción de la entrega

Compilar e instalar

Abrir V3Integracion1.0 en Android Studio. El proyecto requiere JDK 21 para Gradle, SDK Android 37 y 35 y Android 8/API 26 o superior para instalar. El código de las bibliotecas se compila con objetivo Java 17.

```
.\gradlew.bat :app:assembleDebug
.\gradlew.bat :app:testDebugUnitTest
.\gradlew.bat :audioToHaptics:testDebugUnitTest
.\gradlew.bat :app:connectedDebugAndroidTest
```

APK generado: app/build/outputs/apk/debug/app-debug.apk. Las pruebas conectadas requieren un emulador o teléfono con depuración habilitada. No se necesita generar Demucs para compilar la entrega actual.

Usar Cotidiano

1. Abrir Cotidiano y seleccionar un contexto.
2. Revisar los sonidos disponibles y los pendientes.
3. Pulsar Iniciar escucha y conceder el permiso de micrófono.
4. Para usar el brazalete, elegir su firmware y conectar la háptica.
5. Cambiar de contexto detiene la sesión; iniciar de nuevo para el nuevo modo.
6. Probar patrón ejecuta una demostración manual y no añade una detección al historial.

Usar Música ahora y después

Ahora se puede seleccionar y reproducir un archivo local. El panel explica la ausencia de Demucs. Si el equipo entrega el modelo validado, colocarlo en models/htdemucs_6s.onnx y recompilar: el módulo opcional verificará su SHA-256 antes de empaquetarlo.

La huella esperada es:
`16ebcff7c392fc1a21bd2574ea686d20c0ffd9abfd7dcb56680359cb5450c0e0`

Con ambos modelos presentes seguirá siendo necesario elegir explícitamente los parámetros de prueba o incorporar una configuración de hardware confirmada en una etapa posterior. Copiar el modelo al teléfono sin recompilar esta aplicación no activa el paquete de assets.

INFORME TÉCNICO / 15

Mapa técnico y pendientes

Archivo o carpeta	Responsabilidad
app/.../integracion/RegistroAlgoritmos.kt	Fábricas de las bibliotecas reales.
app/.../integracion/IntegracionViewModel.kt	Sesiones, eventos, salida cotidiana, API musical y vista previa.
app/.../integracion/CatalogoCotidianoEquipo.kt	Catálogo de interfaz derivado de los assets.
app/.../ui/screens/PanelesEquipos.kt	Paneles cotidiano y musical.
app/.../ui/screens/ExperienceScreen.kt	Navegación y reproductor de la app.
sound-engine/src/main/	Motor cotidiano y sus cuatro assets.
audioToHaptics/src/main/	Pipeline musical y Basic Pitch, conservados.
docs/CAMBIOS_INTEGRACION.md	Registro de qué cambió, dónde y por qué.
docs/VALIDACION.md y resultado_pruebas.json	Resultados y límites de las comprobaciones.
docs/auditoria/	Hashes, resumen y diferencias de fuentes.

En las rutas app/... se omite el prefijo src/main/java/com/rommelguerra/prototipomusichaptic para facilitar la lectura. El script que generó este informe está en scripts/generar_informe_pdf.py.

Pendientes que se conservaron expresamente

Demucs sigue ausente; alarma de humo, timbre y nombre del usuario siguen sin entrenar; maquinaria y vidrio roto no tienen un comando de firmware nuevo; los parámetros físicos de actuadores siguen sin confirmar; y no existe transmisión musical de tres canales al ESP32.

Las siguientes etapas requieren recursos y pruebas adicionales: modelo validado, teléfono físico para memoria y tiempos, firmware compatible con canales musicales y verificación de latencia y respuesta mecánica. Nada de ello se presenta como completado por compilar una APK o superar pruebas de emulador.

Resultado verificable: una aplicación compilable que usa el motor cotidiano recibido, contiene la biblioteca musical y muestra con precisión sus requisitos pendientes. Los cambios de integración están separados de los algoritmos y respaldados por pruebas y una comparación de fuentes.